

[Chap 13] 순환신경망과 시계열 예측

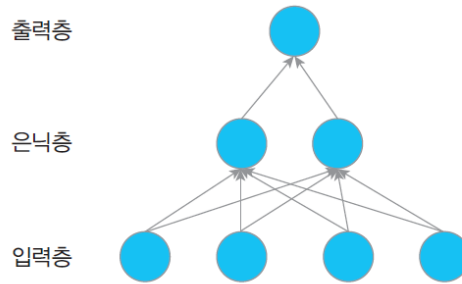


시계열 데이터(time-series data)와 RNN

- 시계열 데이터: 시간 순서에 의존하는 자료
 - 주식가격, 금리, 제품 수요, 기온, 혈압, 음성신호 ...
 - 언어로 된 텍스트도 시계열 자료에 해당
 - 앞에서부터 글자가 순차적으로 나타나서 그 다음 글자의 해석에 영향을 미침
- 시간 순서의 관찰은 본질적으로 과거 데이터에 대한 '기억'에 의존
 - 주가 예측을 하려면 2주 전에 3만 원이었던 주식이 지금 5만 원까지 온 과정을 '기억'해야만 그 다음 주가를 예측할 수 있음
 - 단순한 신경망 구조에서는 시간에 따른 기억 유지를 할 수 있는 장치가 없음
- RNN(순환신경망, Recurrent Neural Network)
 - 시간에 따른 기억을 유지하고 이를 바탕으로 시계열 데이터를 해석하고 예측하기 위해 만들어진 신경망 구조

전통적인 신경망

- 피드포워드 신경망 (Feed-forward Neural Network)
 - 입력으로부터 단방향으로 순차적으로 은닉층을 거쳐 출력값 계산
 - 그 다음 데이터가 들어왔을 때 이전의 데이터가 무엇이었던지를 기억하는 장치가 없음

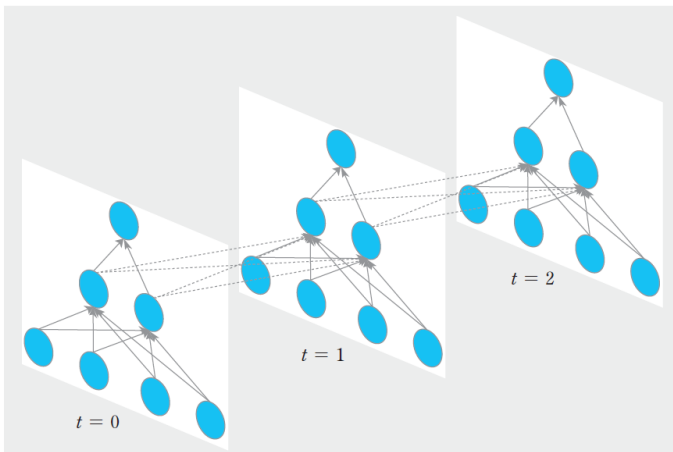


[그림 13-1] 단순한 인공신경망의 예

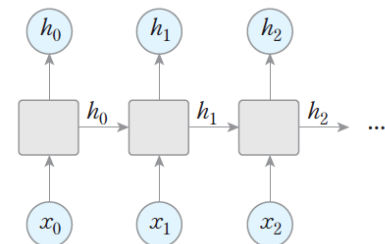
3

RNN

- 다음 입력이 들어왔을 때 입력 데이터뿐 아니라 이전에 가지고 있었던 은닉층의 출력값도 함께 입력으로 고려
 - 기존 은닉층의 출력값을 은닉 상태(hidden state)라고 함



[그림 13-2] RNN의 개념



[그림 13-3] RNN의 도식화

- RNN의 학습: BPTT (Back-Propagation Through Time)
 - 역전파를 시간 축으로 거슬러 올라가도록 확장

4

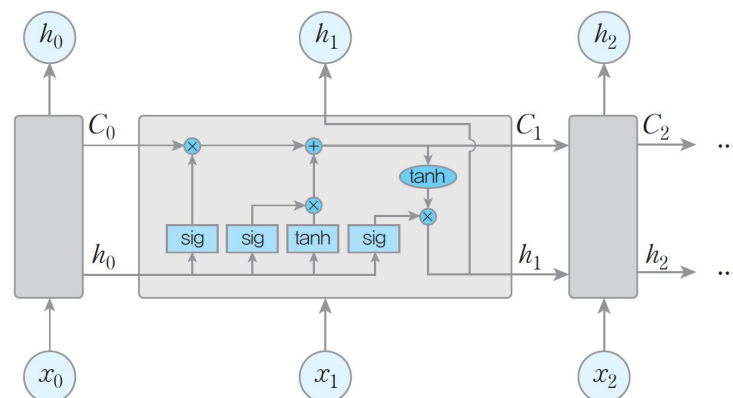
단순 RNN의 한계

- 그래디언트 소실에 의한 장기 의존성 문제(long-term dependency)
 - 단순한 RNN은 시간축을 거슬러 올라가면서 역전파하는 과정에서 일정 기간 이상에서는 그래디언트가 0이 되어 먼 과거 데이터를 고려하지 못함
- 오래전에 나온 데이터가 나중에 중요해지는 경우가 많음
 - 비가 와서 ()을 펼쳤다 → 바로 앞의 '비'에서 괄호 안의 '우산'을 쉽게 유추
 - 나는 부산에서 어린 시절을 보냈다 ... 지금도 () 사투리는 영 고쳐지지 않는다
→ 한참 이전에 나온 '부산'이라는 정보를 기억해야만 괄호 안의 '부산'을 유추 가능
- 과거 기억이 중요한 경우가 있으므로, 단순한 RNN은 한계가 있음
 - 단기기억뿐 아니라 장기기억도 함께 유지하도록 개선된 RNN 모델들이 제시
 - LSTM(Long-Short Term Memory), GRU(Gated Recurrent Unit) ...

5

LSTM(Long Short Term Memory)

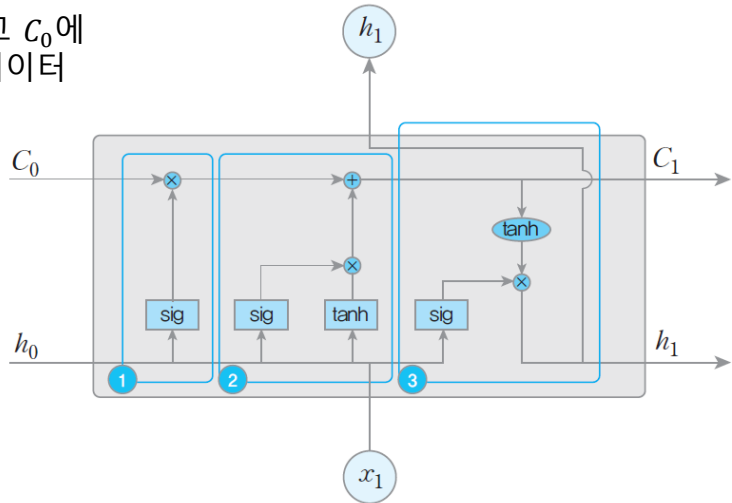
- 장기기억을 유지하면서 필요하면 망각도 할 수 있는 메커니즘 구현
 - 시간을 따라 넘어가는 링크가 두 개임
 - 아래쪽에 있는 링크는 RNN과 마찬가지로 직전의 은닉 상태(hidden state)
 - 위쪽에 추가된 링크는 셀 상태(cell state)로서 기억을 장기적으로 전달할 수 있도록 함



6

LSTM의 작동

- 1 : 망각 게이트(Forget gate)
 - 셀 상태(C_0)에 시그모이드 출력값(sig)을 곱하여 필요없는 장기기억 삭제
- 2 : 입력 게이트(Input gate)
 - 새로운 정보를 셀 상태에 추가함
- 3 : 출력값 계산
 - 셀 상태(C_0)는 그대로 내보내고 C_0 에 직전 은닉상태 h_0 와 새로운 데이터 x_1 을 반영하여 출력 생성



[그림 13-5] LSTM의 작동

LSTM을 통한 주식가격 예측 예제

- FinanceDataReader 패키지 설치 (pip install finance_datareader)
 - 종목 티커(ticker) 또는 코드(KOSPI 코드 6자리)로 주가 자료 가져오기 지원

```
C:\> 명령 프롬프트
Microsoft Windows [Version 10.0.19044.1826]
(c) Microsoft Corporation. All rights reserved.

C:\WINDOWS\system32>pip install finance_datareader_
```

- 예) 2010년부터 2019년까지 코카콜라 주가(티커 KO) 가져오기
 - 날짜 이외에 종가(Close), 시가(Open), 고가(High), 저가(Low), 거래량(Volume) 및 전일대비 가격변동(Change) 표시

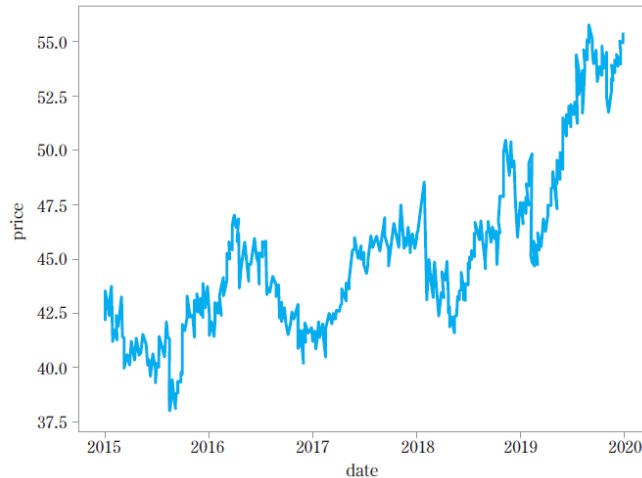
```
1 import FinanceDataReader as fdr
2 stock = fdr.DataReader('KO', '2010-01-01', '2019-12-31')
3 print(stock)
```

	Close	Open	High	Low	Volume	Change
Date						
2015-01-02	42.14	42.26	42.40	41.80	9920000.0	-0.0019
2015-01-05	42.14	42.69	42.97	42.08	26290000.0	0.0000
2015-01-06	42.46	42.41	42.94	42.24	16900000.0	0.0076
...	...	...	...	...	...	...
2019-12-31	55.35	55.20	55.38	54.98	7980000.0	0.0014

[1258 rows x 6 columns]

종가(Close) 시각화

```
1 import numpy as np
2 prices = np.asarray(stock['Close'], dtype='float32')
3 import matplotlib.pyplot as plt
4 plt.figure()
5 plt.plot(stock.index, prices)
6 plt.xlabel('date')
7 plt.ylabel('price')
8 plt.show()
```



9

데이터의 정규화

- 입력 데이터를 0과 1 사이의 값이 되도록 조정
 - 신경망의 입력은 0과 1 사이의 범위로 조정해야 안정적 동작

$$y' = \frac{y - \min(y)}{\max(y) - \min(y)}$$

```
1 y = (prices - min(prices))/(max(prices)-min(prices))
2 print(y)
```

```
[0.23340821 0.23340821 0.25140595 ... 0.97637784 0.9718785 0.97637784]
```

시계열 데이터 준비

- 직전 60일 종가로 다음날 주가를 예측하고자 함
- 만약 1000건의 자료가 있다면, 데이터 포인트의 수는 $(1000 - 60)$ 이 될 것임

첫번째 데이터	0	1	2	...	59	→	60
두번째 데이터	1	2	3	...	60	→	61
세번째 데이터	2	3	4	...	61	→	62
...							
940번째 데이터	939	940	941	...	998	→	999

[그림 13-7] 시계열 학습을 위한 데이터 구조

11

시계열 데이터 준비 (계속)

- 일차원 배열로 되어 있는 종가 데이터를 시계열 학습을 위한 구조로 변환

```
1 window_size = 60
2 data_length = len(y) - window_size
3 y_data = np.zeros(data_length, dtype='float32')
4 x_data = np.zeros((data_length, window_size, 1), dtype='float32')
5 for i in range(0, data_length):
6     y_data[i]=y[i+window_size]
7     x_data[i]=y[i:i+window_size].reshape(window_size,1)
```

- (1~2행) window_size=60, data_length =len(y) - window_size로 둠
- (3행) 정답값을 data_length 길이의 배열로 하여 0으로 초기화
- (4행) 입력 데이터의 구조를 정하여 초기화
 - Keras의 LSTM은 각 데이터 건마다 (기간 수 × 특성치 갯수)의 구조를 입력으로 받음
 - 기간이 60일이고 특성치는 종가 하나 뿐이므로, 각 데이터 건마다 (60×1) 의 형태가 되고, 전체 데이터는 $(data_length \times 60 \times 1)$ 의 3차원 배열 구조
- (5~7행) 데이터 포인트마다 정답값과 입력값 지정
 - 정답값 y_data[i]는 60일 뒤의 주가 y[i + window_size]로 함
 - 입력값 x_data[i]는 i부터 i+59일까지의 주가를 넣음
 - x_data[i]는 (window_size, 1)의 2차원이어야 하므로 reshape(window_size, 1) 적용

12

시계열 데이터 준비 (계속)

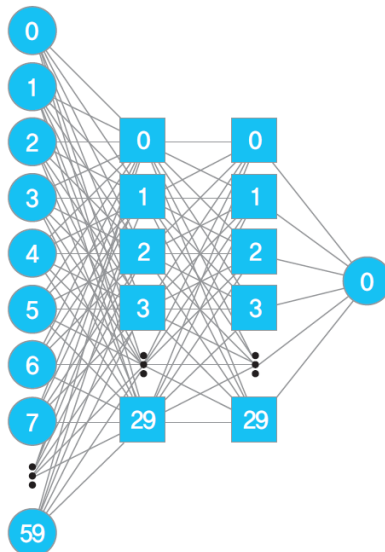
- 학습 데이터와 테스트 데이터로 나눔
 - 30%를 테스트 데이터로 사용
 - 시계열 데이터는 순서가 중요하므로, 테스트 데이터를 임의로 추출하지 않도록 `shuffle=False` 옵션 지정

```
1 from sklearn.model_selection import train_test_split
2 x_train, x_test, y_train, y_test = train_test_split(x_data, y_data,
3   test_size=0.3, shuffle=False)
```

13

신경망 모델링

- 신경망 구조 설계
 - 입력 60개, 출력 1개
 - 은닉층은 2계층으로, 각각에 30개의 LSTM을 두도록 구성



[그림 13-8] 주가 예측을 위한 인공신경망 구조

Keras 모델링 코드

- 앞의 구조를 Keras로 모델링

```
1 model = keras.Sequential([
2     keras.layers.LSTM(30, return_sequences = True, input_shape=(window_size, 1)),
3     keras.layers.LSTM(30),
4     keras.layers.Dense(1, activation=None),
5 ])
```

- (2행) 30개의 LSTM을 두되, 다음 계층으로 60일간의 은닉상태 각각의 순차적인 출력값을 모두 넘겨주어야 하므로, return_sequences=True 옵션 추가
 - (3행) 두 번째 LSTM을 30개로 지정
 - (4행) 최종 출력층은 단일값이고, 연속값을 출력해야 하므로 활성화 함수 없음으로 지정 (activation=None)
- 최적화 알고리즘은 Adam, 출력값이 연속값이므로 손실함수는 'mse' (mean square error)로 지정

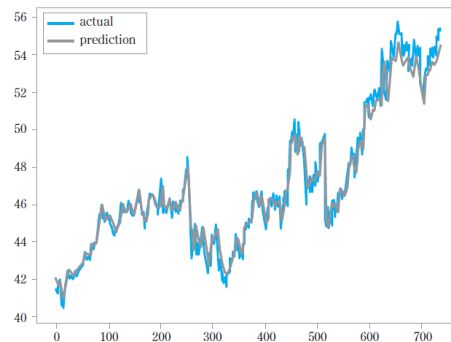
```
model.compile(optimizer='adam', loss='mean_squared_error')
model.fit(x_train, y_train, epochs=30)
```

15

예측 및 시각화

- 예측 수행 및 시각화 (전체 코드: ex13-1.py)

```
28 pred = model.predict(x_test)
29
30 pred_price = (max(prices)-min(prices))*pred + min(prices)
31 target_price = (max(prices)-min(prices))*y_test + min(prices)
32
33 plt.figure()
34 plt.plot(target_price, label='actual')
35 plt.plot(pred_price, label='prediction')
36 plt.legend()
37 plt.show()
```



[그림 13-9] LSTM 기반 인공지능망의 주가 예측 결과

- 위의 예측결과는 직전 60일 주가로 바로 다음날 주가를 예측한 것이므로 기본적으로 추세를 따라가도록 나타남을 주의
- 실제 예측을 위해서는 기업의 경영성과나 시장 반응, 경제상황 등에 대한 다양한 정보를 함께 고려하는 정교한 모델이 필요

전체 코드

예제 ex13-1.py

```
1 import FinanceDataReader as fdr
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 stock = fdr.DataReader('KO', '2010-01-01', '2019-12-31')
6 prices = np.asarray(stock['Close'], dtype='float32')
7 y = (prices - min(prices))/(max(prices)-min(prices))
8
9 window_size = 60
10 data_length = len(y) - window_size
11 y_data = np.zeros(data_length, dtype='float32')
12 x_data = np.zeros((data_length, window_size, 1), dtype='float32')
13 for i in range(0, data_length):
14     y_data[i]=y[i+window_size]
15     x_data[i]=y[i:i+window_size].reshape(window_size,1)
16 from sklearn.model_selection import train_test_split
17 x_train, x_test, y_train, y_test = train_test_split(x_data, y_data,
18     test_size=0.3, shuffle=False)
19
20 from tensorflow import keras
21 model = keras.Sequential([
22     keras.layers.LSTM(30, return_sequences = True, input_shape=(window_size, 1)),
23     keras.layers.LSTM(30, input_shape=(window_size, 1)),
24     keras.layers.Dense(1, activation=None),
25 ])
26 model.compile(optimizer='adam', loss='mean_squared_error')
27 model.fit(x_train, y_train, epochs=30)
28 pred = model.predict(x_test)
29
30 pred_price = (max(prices)-min(prices))*pred + min(prices)
31 target_price = (max(prices)-min(prices))*y_test + min(prices)
32
33 plt.figure()
34 plt.plot(target_price, label='actual')
35 plt.plot(pred_price, label='prediction')
36 plt.legend()
37 plt.show()
```